TECHNOLOGY STACK DOCUMENT

Healthcare Financial Management & Revenue Cycle Management Platform

Industrial-Grade, Zero-License-Cost MVP | Java Backend | Version 2.0

1. Purpose

This document defines the recommended technology stack for the MVP. The objective is to build a serious, production-capable healthcare-finance platform without paying for unnecessary software licenses or introducing infrastructure complexity before it is justified. The stack uses mature open-source technologies, a modular-monolith architecture and a clear path to future scale.

2. Selection Principles

- Zero mandatory software/license cost for MVP development.
- Enterprise-grade maturity and long-term maintainability.
- Simple enough for a small development team.
- Strong transaction, security and data-integrity support.
- Good support for healthcare, insurance and financial integrations.
- Local development must be possible without paid cloud services.
- Avoid premature microservices, Kubernetes and distributed infrastructure.
- Keep technology choices replaceable where practical to reduce vendor lock-in.

3. Recommended Stack — At a Glance

Layer	Technology	Purpose	MVP Cost	Decision
Backend	Java 21 LTS + Spring Boot	Core application/API layer	₹0 software license*	Recommended
API	Spring Web + OpenAPI	REST APIs and contracts	₹0	Recommended
Security	Spring Security	Authentication/authorization	₹0	Recommended
Validation	Jakarta Bean Validation	Input/domain validation	₹0	Recommended
Persistence	Spring Data JPA + Hibernate	Database persistence	₹0	Recommended
Database	PostgreSQL	Transactional source of truth	₹0	Recommended
Migrations	Flyway	Version-controlled schema changes	₹0	Recommended
Frontend	React + TypeScript	Web UI	₹0	Recommended
Async/Cache	Redis	Cache and limited async needs	₹0	Recommended
Background Jobs	Spring scheduling / async; add queue only when needed	Background processing	₹0	Recommended
Documents	MinIO / S3-compatible storage	Claim/auth documents	₹0	Recommended
Build	Maven	Build/dependency management	₹0	Recommended
Testing	JUnit 5 + Mockito + Testcontainers	Automated testing	₹0	Recommended
Containers	Docker Engine + Compose	Local/deployment packaging	₹0*	Recommended
Reverse Proxy	Nginx	Routing/TLS termination	₹0	Recommended

Quality	Checkstyle/SpotBugs + ESLint	Code quality	₹0	Recommended
CI	GitHub Actions or GitLab CI	Build/test automation	Free tiers vary	Recommended
Observability	Structured logs + OpenTelemetry-ready design	Diagnostics	₹0	Start simple

4. Why Java + Spring Boot?

Java is selected for the core backend because this product is primarily a transactional enterprise system, not an AI-first application. Billing, claims, payments, reconciliation, authorization and external integrations require strong structure, security, transaction management and long-term maintainability.

- Enterprise maturity: Java/Spring is a proven choice for large transactional systems.
- Strong typing: compile-time checks help reduce errors in financial workflows.
- Transaction support: Spring's ecosystem works naturally with relational transactions and PostgreSQL.
- Security: Spring Security provides a mature foundation for authentication and authorization.
- Integration fit: Spring is well suited to REST APIs and external-system adapters.
- Maintainability: clear module boundaries and dependency injection support a modular monolith.
- Future scale: the same codebase can grow before services need to be separated.
- AI does not justify Python in the core backend: future ML functionality can be exposed as a separate service if it becomes necessary.

5. Java Version Decision

Use Java 21 LTS as the baseline for the MVP if the team has already standardized on it. Java 21 is an LTS release. However, because the current date is August 2026 and Oracle's roadmap states that Oracle JDK 21 updates after September 2026 are planned to move from the no-fee terms to the Oracle Technology Network license, the project should standardize on a permissively licensed OpenJDK distribution or evaluate Java 25 LTS before production. Oracle identifies Java 25 as the current LTS release and states that it is available under its no-fee terms. □cite□turn0search0□turn0search16□

Practical decision: Java 21 LTS is acceptable for development if the team already uses it; for a new long-lived project, evaluate Java 25 LTS before locking the production runtime. Do not assume 'Java = free' means every Oracle JDK update is permanently free.

6. Spring Boot

Spring Boot is the recommended application framework. Current Spring Boot documentation supports modern Java versions and Maven/Gradle builds. □cite□turn0search1□

- Spring Web: REST API layer.
- Spring Security: authentication and authorization.
- Spring Data JPA: persistence integration.
- Spring Validation: request validation.
- Spring Actuator: health and operational endpoints.
- Spring testing support: unit and integration testing.

7. PostgreSQL

PostgreSQL remains the single authoritative database for financial transactions. Its official documentation lists transactional integrity, foreign keys, concurrency control and complex queries among its capabilities, and its license permits free commercial use. □cite□turn0search4□turn0search6□

- Suitable for invoices, claims, payments and reconciliation.
- Strong relational consistency.
- Constraints protect financial data integrity.
- One database avoids unnecessary distributed-data complexity.
- Open-source and commercially usable without a license fee.

8. Persistence and Database Migration

- Spring Data JPA + Hibernate: standard persistence layer.
- Flyway: schema migrations checked into source control.
- Use database constraints for critical invariants, not only Java validation.
- Use explicit transaction boundaries around financial operations.
- Do not put business rules exclusively inside ORM mappings.

9. Frontend — React + TypeScript

- React provides reusable components for dashboards, forms and workflow screens.
- TypeScript provides compile-time type safety.
- Good fit for role-based operational screens and financial reporting.
- Separates UI concerns from the Java backend.
- Free/open-source development stack.

10. Redis and Background Processing

Redis is recommended only where it solves a real problem: caching, temporary state or lightweight asynchronous coordination. For MVP background work, use Spring's scheduling/async capabilities where sufficient. Introduce a dedicated message broker only when requirements justify it.

- PostgreSQL remains the source of truth.
- Do not store authoritative financial state only in Redis.
- Do not add Kafka/RabbitMQ on day one merely because the architecture may eventually need events.

11. Document Storage — MinIO

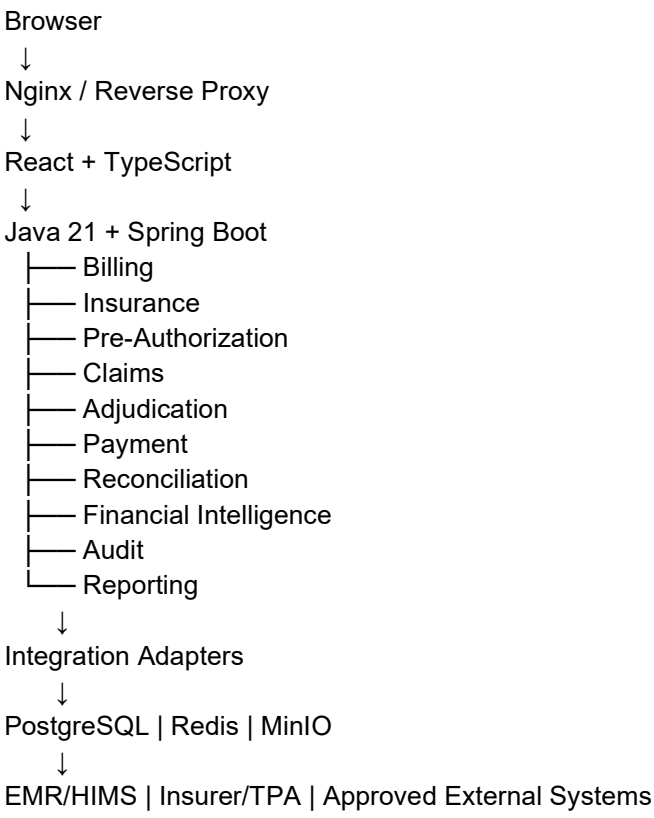
- Store large claim, authorization and supporting documents outside PostgreSQL.
- Keep document metadata and relationships in PostgreSQL.
- Use an S3-compatible interface to preserve future migration options.
- Self-hosted/local MinIO allows MVP development without paid cloud storage.

12. Docker and Deployment

Use Docker Engine and Docker Compose to run the application and supporting services consistently. Docker Engine is open-source; however, Docker Desktop has separate subscription terms for certain larger organizations, so the team should distinguish Docker Engine from Docker Desktop when enforcing the zero-cost requirement. □cite□turn0search12□turn0search7□

- Local development: Docker Compose.
- CI: containerized build/test.
- Initial deployment: simple Linux host/container environment.
- Do not introduce Kubernetes until operational scale actually requires it.

13. Recommended Architecture



14. Architecture Style

- Modular monolith for MVP.
- DDD-aligned domain modules.
- Clean/Hexagonal boundaries between domain, application, infrastructure and integrations.
- REST APIs for synchronous operations.
- Events/background jobs only where they provide a clear benefit.
- One PostgreSQL database initially.
- External systems accessed through adapter interfaces.

15. MVP vs Future Scaling

Area	MVP	Future
Backend	Java + Spring Boot modular monolith	Split services only when justified
Database	One PostgreSQL	Read replicas/partitioning if required
Async	Spring async/scheduling + Redis where useful	Kafka/RabbitMQ or dedicated event platform if justified
Storage	MinIO / S3-compatible	Managed object storage if justified
Deployment	Docker Compose / simple container host	Orchestration platform if justified
Observability	Structured logs + health/metrics	Centralized observability stack
AI	Basic/rule-based financial intelligence	Separate Python/ML service if genuinely needed

16. What We Deliberately Avoid

- Microservices in MVP — too much distributed-system overhead.
- Kubernetes — operationally excessive for initial scale.
- Kafka/RabbitMQ from day one — unnecessary until event throughput/decoupling requires it.

- Multiple databases — PostgreSQL is enough initially.
- Paid API gateways — Nginx plus Spring Security is sufficient for MVP.
- Paid monitoring — start with open-source logs/metrics.
- Cloud-specific database services — avoid lock-in and recurring cost.
- Separate Python backend — unnecessary; use Python later only for genuinely AI/ML-specific services.
- Premature service mesh — adds complexity without MVP value.

17. Security Baseline

- HTTPS/TLS in deployed environments.
- Spring Security for authentication/authorization.
- Role and permission checks.
- Tenant isolation at service and data-access boundaries.
- Secure password hashing if credentials are locally managed.
- Secrets outside source control.
- Audit logging for material financial and authorization events.
- API validation and centralized exception handling.
- Least-privilege database users.
- Dependency/container vulnerability scanning in CI.

18. Testing Baseline

- JUnit 5 — unit tests.
- Mockito — isolated dependency mocking where useful.
- Spring Boot Test — application/integration tests.
- Testcontainers — real PostgreSQL/Redis-style integration tests in containers.
- React Testing Library — frontend behavior tests.
- Critical financial workflows must have automated tests before MVP release.
- Test idempotency, transaction rollback, duplicate requests and failure/retry paths.

19. Developer Implementation Baseline

- Java 21 LTS initially; evaluate Java 25 LTS for the long-lived production baseline.
- Spring Boot + Spring Web + Spring Security + Spring Data JPA.
- PostgreSQL as authoritative transactional database.
- Flyway for migrations.
- React + TypeScript frontend.
- Redis only for justified caching/temporary async needs.
- MinIO/S3-compatible storage for documents.
- Maven for builds.
- JUnit 5 + Testcontainers for backend testing.
- Docker Compose for local development.
- Nginx for reverse proxy.
- Git + CI pipeline.
- Modular monolith with DDD/Clean/Hexagonal boundaries.

20. Cost Model

The goal is zero mandatory software-license cost for MVP development, not a claim that production will permanently cost ₹0. Hosting, domain, backups, email/SMS, external payer APIs and managed infrastructure can cost money.

Area	MVP Approach	Software License Cost
Java runtime	OpenJDK/permissively licensed distribution	₹0
Spring Boot	Open source	₹0
PostgreSQL	Open source	₹0
React/TypeScript	Open source	₹0
Redis	Open source	₹0
MinIO	Open-source/self-hosted option	₹0 software license
Docker Engine	Open source	₹0
Nginx	Open source	₹0
Testing/build tools	Open source	₹0
CI	Free tier where applicable	₹0 initially
Production hosting	Deferred until deployment decision	Not included
External integrations	Mocks/stubs during MVP where possible	Provider-dependent

21. Final Recommendation

Use Java + Spring Boot as the core backend. Pair it with React + TypeScript, PostgreSQL, Flyway, Redis where justified, MinIO, Docker and Nginx. Keep the system as a modular monolith. This is the better balance for this healthcare-finance product: strong enterprise foundations without paying for or operating unnecessary distributed infrastructure.

The important constraint is discipline: industrial-level does not mean adding every enterprise technology. The MVP should earn its complexity through real requirements.

22. Next Step

After approval of this technology stack, review the BRD and LLD against the Product Structure and this stack. Produce the requirement-to-module mapping, API list, database entities, integration dependencies and MVP development backlog before implementation begins.

Current-source note: Java licensing/support statements were checked against Oracle's August 2026 roadmap; Spring Boot requirements were checked against current Spring documentation; PostgreSQL and Docker licensing statements were checked against official project documentation.